

Kiss Assembly Language Manual

License Notice Copyright © 2026 Wolven LLC. This document is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). See [docs/Licenses/CC-BY-4.0.txt](#) and <https://creativecommons.org/licenses/by/4.0/>.

Version 1.0 Draft

1. Introduction

1.1 Purpose

This manual describes the Kiss assembly language from the perspective of a programmer who is reading, writing, or maintaining Kiss assembler source. It covers source-line structure, literal and label syntax, operand notation, modifier suffixes, assembler directives, macros, conditional assembly, and multi-module linking.

The goal is to give the programmer a single place to learn how Kiss source files are organized, what the assembler will and will not accept, and how common idioms are expressed. It is neither a complete architectural reference nor a per-command cookbook; those roles belong to companion documents.

1.2 Scope

This manual stays at the **assembler surface**. It describes the language the programmer types into `.kasm` source files and the meaning of each lexical element at that level.

Topics intentionally left to other documents:

- **ISA architecture.** Register banks, 100-bit instruction layout, CMB encodings, CEB state, ALU mechanics, privilege levels, and all hardware-facing details belong to [docs/isa/kiss-isa-reference.md](#) (authoritative) and [docs/isa/kiss-isa-manual-source.md](#) (human-readable).
- **Per-command detail.** The command-by-command reference — full mnemonic inventory, operand tables, CMB layouts for each family — lives in [docs/assembler-language/kiss-assembler-commands-source.md](#).
- **Formal grammar.** The working grammar specification from which this manual draws its structural rules is [docs/assembler-language/kiss-assembler-grammar.md](#).

Assembler-level detail that the command reference does not repeat — line shape, literal forms, labels, directives, macros, conditional assembly — lives here.

1.3 Related documents

Document	Role
<code>docs/isa/kiss-isa-reference.md</code>	Authoritative ISA specification. When two documents disagree, this wins.
<code>docs/isa/kiss-isa-manual-source.md</code>	Human-readable ISA manual covering architecture and command families at the ISA level.
<code>docs/assembly-language/kiss-assembly-commands-source.md</code>	Per-command assembler reference: every mnemonic, its operand pattern, CMB modifier bits, and examples.
<code>docs/assembly-language/kiss-assembly-grammar.md</code>	Working grammar specification for Kiss assembler source. This manual is the prose companion to that spec.
<code>docs/isa/kiss-isa-operand-notation.md</code>	Complete reference for operand-role notation (R, sR, dR, iR, asR, adR, optional o prefix).

2. Conventions

These conventions apply to the whole manual. Mnemonic-level conventions (canonical casing, suffix ordering rules as applied inside a single mnemonic) are covered in more depth in §2 of the commands reference; what follows is the file- and program-level view.

2.1 Case sensitivity and whitespace

Kiss assembler source is **case-sensitive**. Mnemonics, directives, register names, and labels must match their canonical casing exactly. Whitespace is insignificant except as an operand separator and as a line terminator for line comments.

2.2 Line structure

Source is processed one line at a time. A single line contains at most one of:

- a label declaration
- an instruction
- a directive
- a comment

Blank lines are ignored. Operand separation is by whitespace only — Kiss assembler does **not** use commas between operands.

2.3 Comment styles

Kiss supports both line comments and block comments:

Style	Opens	Closes	Notes
Line comment	//	end of line	May follow any instruction, label, or directive on the same line.
Block comment	/*	*/	May span multiple lines. Nested block comments are not supported.

2.4 Canonical mnemonic casing

ISA command names use mixed-case canonical forms: **Move**, **SetVal**, **ChkBnds**, **Compare**, **Test**, **Retrn**. The older short or all-caps forms (**Mov**, **Tst**, **Cmp**, **CHKBND**, **SETIMM**) are **deprecated** and are not accepted by the current assembler. For the full mnemonic-casing rules, suffix ordering, and dotted-prefix rules as applied inside a single mnemonic, see §2.1 of the commands reference.

2.5 1-based indexing, 10-bit bytes, 100-bit instructions

Kiss uses **1-based indexing** throughout: section numbers, register IDs, bit positions within a register, operand slots (1-5), and array indices all start at 1. Zero is the universal invalid sentinel.

A **Kiss byte is 10 bits**, not 8. Byte counts in Load/Store, Read/Write, and similar families are in 10-bit bytes. An instruction is exactly **100 bits** wide, which is ten Kiss bytes.

3. Source-line shape

3.1 Anatomy of a line

A full source line has this general shape:

```
[<label>] <mnemonic-or-directive> [<operand1> <operand2> ... <operandN>] [// line comment]
```

Any of the brackets may be empty. A bare label occupies its own line. A bare block-comment line has no other content. A line with only a line comment has no mnemonic.

Examples:

```
@LoopTop           // label declaration
    I.Add R1A R1B R1C // instruction
    JmpNZ @LoopTop   // instruction with label target
Def Counter Bin 4    // directive
// entire line is a comment
```

3.2 Whitespace rules

- One or more spaces or tabs separate a mnemonic from its operands.
- Operands are separated by whitespace only. Commas are not used.
- Leading indentation is stylistic; the assembler does not enforce it.

- No Kiss mnemonic or directive contains internal whitespace.
- Blank lines are ignored.

3.3 Comment placement

- A line comment (`//`) may appear at the end of any non-comment line, or on a line by itself.
- A block comment (`/* ... */`) may appear on its own line(s), or inline between tokens where whitespace would be legal. In practice it is clearest as a standalone documentation block above the code it describes.
- Comments are stripped before the parser sees the line. They do not affect tokenization of adjacent operands.

4. Literals and type tags

Kiss source recognizes **bare numeric literals** and **typed literals**. Typed literals are introduced by `#` followed by a one-letter type tag and a colon.

The current type tags are:

Tag	Form	Type
<code>#B:</code>	<code>#B:<decimal-number></code>	Binary (unsigned)
<code>#I:</code>	<code>#I:<signed-decimal></code>	Integer (ones-complement signed)
<code>#D:</code>	<code>#D:<decimal-number></code>	Decimal (packed 5-bit digits)
<code>#T:</code>	<code>#T:"<text>"</code>	Text (10-bit Kiss character codes)
<code>#:</code>	<code>#:<value></code>	Untyped — valid only inside Def directives

4.1 Numeric literal types

`#B:` introduces a binary (unsigned) value. The digits after the colon are read as decimal text and the assembler converts the value to straight binary.

```
SetVal R1A #B:42
I.AddTo #B:1 Counter
```

`#I:` introduces an integer. Integers are signed in ones-complement. A leading `-` marks a negative integer.

```
SetVal R1A #I:-7
I.Add R1A R1B #I:1000 R1C
```

`#D:` introduces a decimal value. Decimal literals may include an embedded decimal point and a leading `-`.

```
SetVal TaxRate #D:8.25
D.Add Balance #D:-12.50 Balance
```

All numeric literal values are entered as **decimal text**. There is no binary, octal, or hexadecimal entry form. The assembler converts the decimal text into the appropriate internal representation (straight binary, ones-complement, or packed 5-bit digits).

4.2 Text literals

Text literals carry the tag **#T:** and a double-quoted body:

```
SetTxt UserPrompt #T:"Enter user name: "  
Def AppTitle Txt 30 #:"Kiss OS"
```

The current Lexer accepts the standard double-quote delimiters ("**...**") and recognizes the following backslash escapes inside the quoted body:

Escape	Meaning
\ "	literal double quote
\\	literal backslash
\n	newline
\t	tab

Any other **\x** sequence produces the literal character **x**. A quoted string may not span a line — a newline inside quotes is a lexer error.

Note — upcoming guillemet delimiters. The 2026-03-29 architecture decision adopts «**...**» (Kiss character-map positions 47 and 48) as the future text-literal delimiters, replacing the "**...**" form. The Lexer has not yet been updated; all examples in this manual use the current "**...**" form. When the transition lands, the Lexer, Preprocessor, tests, this manual, the commands reference, and the VS Code extension grammar will be updated together. See Appendix B for more detail.

4.3 The untyped literal (**#:**)

The bare **#:** form carries a value with no type tag. It is legal **only** inside a **Def** directive, where the **Def** type keyword already determines the data type. Using **#:** outside of **Def** is a syntax error.

```
Def Flags      Bin 2 #:255           // binary initializer  
Def TaxRate    Dec 10 4 #:8.25       // decimal initializer  
Def AppTitle    Txt 30 #:"Kiss OS"    // text initializer
```

4.4 Literal syntax rules

- **Decimal text only.** Numeric values are always written in decimal. No **0x**, **0b**, or octal prefix is recognized.
- **Sign.** A leading **-** is permitted on **#I:** and **#D:** literals. **#B:** is unsigned; a negative binary literal is a syntax error.
- **Decimal point.** For **#D:** (and for a **#:** value inside **Def Dec**), a single **.** marks the decimal point. The **positions** argument on **Def Dec** fixes the number of fractional digits stored; the compiler aligns the literal to that position count.

- **Text body.** The text body inside `#T:"..."` is a quoted string using the escapes listed in §4.2. Each character in the final string is converted to a 10-bit Kiss character code when the literal is emitted.
- **Bare numeric literals.** A token that is not a register ID and does not begin with `#` or `@`, but consists entirely of numeric text, is treated as a **bare numeric literal**. It is interpreted as an unsigned integer/binary value and is the concise form for small counts and port numbers.

```
Load 4 BaseAddr R1A           // 4 is a bare numeric literal (iR operand)
WriteL 8 Port1 R2A           // 8 and Port1 are the byte count and port
```

5. Labels and scope

5.1 Label syntax

Labels in Kiss assembler begin with the `@` character and are followed by an identifier:

```
@Start
@Loop1
@WriteResult
```

The identifier after `@` starts with a letter or underscore and continues with letters, digits, or underscores. The `@` marker is reserved for labels and label references — it is not a general address prefix.

A label declaration occupies its own line:

```
@Main
    SetVal R1A #I:0
    Jmp @ProcessLoop
```

5.2 Local vs global scope

By default a label is **module-local** — it is visible only within the source file (and any files it **Includes**) that define it. Labels become visible across modules only through the **Export / Import** directives described in §13.

5.3 Label as branch target

Labels are the normal target form for control-transfer instructions. The assembler lowers a `@Name` operand to the correct PC-relative or absolute form expected by the instruction:

```
@Retry
    I.Decr #I:1 Counter
    JmpNZ @Retry           // conditional jump to label
    Call @Finalize         // absolute call to label
    FJmp @FastPath         // forward PC-relative jump
```

If* instructions that exit through **ClrCF** — or jump out of a conditional block via a CEB-clearing

jump — also use labels as their targets. The assembler handles the CEB-clear count automatically when the jump crosses conditional boundaries.

5.4 Label as data reference

A label declared ahead of a **Def** directive marks the address of that data:

```
@MessageTable
  Def Greeting Txt 32 #:"Hello, world!"

  LoadL 32 @Greeting R2A
```

The assembler resolves **@Greeting** to the storage address of the **Defed** buffer.

5.5 Export / Import / linker visibility

For a brief taste of multi-module linking (covered fully in §13):

- **Export @Name** makes a label visible to other modules.
- **Import @Name** declares that a label is defined elsewhere and will be resolved by the linker.

```
Export @CalcInterest      // in math.kasm
Import @CalcInterest      // in main.kasm
```

6. Operand categories and notation

6.1 Notation summary

Command reference tables use compact operand-role markers. The full reference for this notation lives in [docs/isa/kiss-isa-operand-notation.md](#); the common markers are:

Marker	Meaning
R	Plain register operand
sR	Source register (read only)
dR	Destination register (written)
sdR	Source-and-destination register (read, then written in place)
iR	Immediate-or-register operand (CMB-3 selects which)
asR	Source-address register (holds memory address; memory is read — Load family)
adR	Destination-address register (holds memory address; memory is written — Stor family)
o prefix	Optional operand (e.g. odR, oaR)

6.2 Position suffixes 1-5

A trailing digit on an operand marker identifies which operand slot it fills. **iR1** is an immediate-or-register in operand 1; **dR5** is a destination register in operand 5; **odR4** is an optional destination

register in operand 4.

6.3 Destination-last convention

Kiss operand order follows a **destination-last** convention:

- Sources fill **forward** from Operand 1.
- The primary destination is **Operand 5**.
- Optional additional destinations (for fan-out families) fill **backward** from Operand 4.
- A Register ID of 0 in any slot means the slot is unused.

```
I.Add R1A R1B R2A           // R2A = R1A + R1B      (sR1 sR2 dR5)
I.Add R1A R1B R1C R1D R2A   // R2A = R1A+R1B+R1C+R1D (sR1..sR4 dR5)
D.Incr #D:1 R1A R1B R1C     // three in-place increments by 1 (iR1 odR4 odR3 dR5 – filled
back-to-front)
```

This convention applies to typed math, logic, compare, shift/rotate, and most data-movement families. Deviations are documented per-command in the commands reference.

6.4 The SetImm exception

The ISA command **SetImm** is the sole documented exception to the destination-last rule. Its destination is **Operand 1** and its 50-bit immediate payload is packed into **Operands 2-5**.

Note. **SetImm** is an ISA-level name. At the assembler surface the programmer writes **SetVal** (for numeric immediates) or **SetTxt** (for text immediates); both lower to the same **SetImm** ISA command, with CMB-4 distinguishing numeric (0) from text (1). **SetImm** itself is **not** an assembler mnemonic.

```
SetVal R1A #I:42           // destination in Operand 1; 50-bit numeric payload
SetTxt AppTitle #T:"Kiss OS" // destination in Operand 1; 50-bit text payload
```

6.5 Fan-out and in-place operand patterns

Families that accept multiple destinations (**fan-out**) — such as **CopyL**, **MoveL**, **AddTo**, **Incr**, **Decr**, **Not**, **Rot**, **Shift** — use optional destination slots **odR2..odR4** in addition to the primary **dR5**. Each trailing destination receives the same result independently.

The **o** prefix on a single-operand in-place form (for example **o*R** markers in certain families) indicates that the operation is applied in place to the named register when a separate destination is not supplied. The commands reference gives the precise operand patterns per family.

Fan-out is **not** available for: **Load***, **Stor***, **Neg**, **Cvt**, **BitOn**, **BitOff**, and **BitFlip**.

7. Dotted datatype prefix

Typed Kiss commands carry a dotted datatype prefix that tells the assembler (and the reader) which ALU processes the instruction.

7.1 B. / I. / D.

A single dotted prefix selects the primary data type:

Prefix	Type	ALU	CMB 1-2
B.	Binary	Binary ALU (adder)	01
I.	Integer	Integer ALU (subtractor)	10
D.	Decimal	Decimal ALU (matrix lookup)	11

B.Add R1A R1B R1C
I.Sub R1A R1B R1C
D.Mul R1A R1B R1C

// Binary add
// Integer subtract (ones-complement)
// Decimal multiply

7.2 Untyped commands

Commands that are not parameterized by a data type carry no dotted prefix and encode **CMB 1-2 = 00**. These include:

- Bit-manipulation family: **BitOn**, **BitOff**, **BitFlip**, **BitTstOn**, **BitTstOff**, **BitRead**, **BitWrite**
- Load / Store family: **LoadL**, **LoadR**, **LoadLOfs**, **StorR**, ...
- I/O family: **ReadL**, **ReadR**, **WriteL**, **WriteR**
- Stack family: **Push**, **Pull**
- Control flow: **NOP**, **Brk**, **Jmp**, **Call**, **Retrn**, **If***, **EndIf**, **ClrCF**

```
Jmp @Exit  
Push R1A  
BitTstOn Cry StatusFlags
```

7.3 Double-dotted for Cvt

Conversion commands carry **two** dotted prefixes: the source type, then the destination type, then **Cvt**:

```
B.I.Cvt R1A R2A      // Binary -> Integer  
I.D.Cvt R2A R3A      // Integer -> Decimal  
D.B.Cvt R4A R5A      // Decimal -> Binary
```

There are no three-dotted forms.

8. Modifier suffixes

A typed base mnemonic can carry one or more suffixes that select family-specific modifiers. Suffixes are written without separators (no dots, no underscores) and must appear in a fixed order.

8.1 Suffix order

```
<TypePrefix> <Base> [<Direction>] [<CarryOrBorrow>] [<Saturation>] [<SetFlags>]
```

For example, **I.AddCrySatSet** decomposes as:

```
I.      = Integer prefix
Add     = base mnemonic
Cry     = include carry input (multi-precision chain)
Sat     = saturating clamp
Set     = set status flags after operation
```

Any unused position is simply omitted. The suffix order is fixed so that the assembler can parse the mnemonic unambiguously from left to right.

8.2 Direction suffixes (L / R)

The direction suffixes are:

Suffix	Meaning	Applies to
L	Left	Rotate, Shift, Copy, Move, Load, Stor, Read, Write
R	Right	same families

Direction is part of the base mnemonic for I/O and Load/Stor families; it is the first suffix position for Shift and Rotate.

```
B.RotL R1A #B:4          // rotate left by 4
I.ShiftR R1A R2A R3A      // shift right
CopyL R1A R2A R3A         // copy left-aligned
LoadL 4 BaseAddr R1A      // load left-aligned
WriteR 8 Port1 R2A        // write right-aligned
```

8.3 Carry / borrow suffix

Suffix	Meaning	Valid on
Cry	Include carry input (multi-precision arithmetic chain)	Add
Brw	Include borrow input	Sub

Cry and **Brw** are **not** valid on **B.Mul** / **I.Mul** / **B.Div** / **I.Div**, on **MulAdd** or **DivRem** of any data type, or on any rotate/shift family (**Rot**, **Shift**, **SShift**).

- **Binary and Integer Mul/Div:** multi-precision multiply/divide uses shift-and-add and long-division algorithms — not single-bit chain. A single-bit Cry/Brw carries no useful chain state.
- **MulAdd:** fused two-stage (Mul → Add) with no clean single-Cry semantic — a chain bit can't

unambiguously mean “Mul-stage carry” or “Add-stage carry.” Multi-precision Decimal multiply uses plain **D.MulCry** + chain math instead.

- **DivRem**: already exposes the remainder via Operand 4, so a separate Brw chain register would be redundant.
- **Rotate / Shift family**: through-carry was retired across the family — the modern RISC consensus is to provide **Add+Cry** / **Sub+Brw** chain math instead.

Cry and **Brw** are valid on **D.Mul** and **D.Div** — Decimal long-multiplication and long-division naturally produce a single-digit carry / borrow per column, and chain through SCRY / SBRW the same way **D.AddCry** / **D.SubBrw** do. **D.MulCry** adds the carry-in digit at the LSD of the product and writes the high overflow digit back to **SCRY**. **D.DivBrw** prepends the borrow-in digit at the high end of the dividend (matching standard long-division) and writes the new remainder digit back to **SBRW**. See the assembler commands reference §8.3.4 for full semantics.

```
I.AddCry R1A R2A R3A
I.SubBrw R1A R2A R3A
D.AddCry R1A R2A R3A
```

8.4 Saturation suffix (Sat)

Sat requests saturating arithmetic: on overflow the result clamps to the maximum representable value; on underflow it clamps to the minimum. Saturation applies to Add, Sub, Mul, Shift, and their carry/borrow variants (where supported).

```
I.AddSat R1A R2A R3A
D.MulSat R1A R2A R3A
I.ShiftLSat R1A #I:12 R2A
```

8.5 Set-flags suffix (Set)

Set requests that the operation update the Status Register’s Arithmetic and Result-property flag groups (Ovr, Und, Wrp, Cry, Brw, Inf, AnyCarry, Z, Pos, Neg, Evn, Odd, THi, TLo). Compare-family operations update the Compare group independently of **Set**.

```
I.AddSet R1A R2A R3A           // add and update arithmetic flags
D.SubBrwSatSet R1A R2A R3A     // sub with borrow, saturate, and update flags
```

Without **Set** the arithmetic flags are left unchanged. This lets a sequence of calculations run without clobbering an earlier comparison’s flag state.

8.6 Combining suffixes

Suffixes combine in the fixed order. A few representative forms:

```
I.AddCrySatSet R1A R2A R3A     // integer add-with-carry, saturating, set flags
D.AddCrySet R1A R2A R3A       // decimal add-with-carry, set flags
B.RotR R1A #B:5               // binary rotate right by 5 bits
I.MulSat R1A R2A R3A          // integer multiply, saturating
```

9. Direction prefix and condition suffixes (Jump / Call / If)

9.1 Direction prefix for Jump and Call

The assembler uses **prefixes F / B** on **Jmp** and **Call** to select PC-relative forward and backward forms. This is the canonical form; the older suffix spellings **JmpF / JmpR / CallF / CallR** have been removed.

Form	Meaning
Jmp	Absolute jump — target is the absolute address in operand 1
FJmp	Forward PC-relative — PC + operand1
BJmp	Backward PC-relative — PC - operand1
Call	Absolute call
FCall	Forward PC-relative call
BCall	Backward PC-relative call

A condition suffix is appended to the whole mnemonic:

```
Jmp    @Done
FJmp   @FastPath
BJmpGT @RetryLoop
CallEQ @HandleMatch
FCallNZ @ProcessNonZero
BCallLE @HandleLowOrEqual
```

9.2 Shared condition suffixes

The following 30 condition suffixes are shared across Jump, Call, and If:

Suffix	Meaning	Suffix	Meaning
Z	Zero	Pos	Positive
NZ	Not Zero	NPos	Not Positive
Blw	Below	Neg	Negative
BlwE	Below or Equal	NNeg	Not Negative
LT	Less Than	EvN	Even
LE	Less Than or Equal	Odd	Odd
EQ	Equal	Ovr	Overflow (directional)
NE	Not Equal	NOvr	Not Overflow
GE	Greater Than or Equal	Und	Underflow (directional)
GT	Greater Than	NUnd	Not Underflow
AbvE	Above or Equal	Brw	Borrow
Abv	Above	NBrw	Not Borrow
Cry	Carry	NCry	Not Carry
THi	High-end truncation	NTHi	No high-end truncation
TLo	Low-end truncation	NTLo	No low-end truncation

The **Blw/BlwE/Abv/AbvE** suffixes test unsigned relationships; **Lt/LE/GT/GE** test signed relationships.

Ovr/Und are *purely directional* — they fire when the mathematical result is above the positive range or below the negative range. **THi/TLo** describe which end of the digit/bit array was truncated and are independent of **Ovr/Und** (the two pairs may co-fire or fire independently). See the grammar §StatusFlags register alias section for the per-operation rule table.

9.3 If-family IfIRQ

The If family adds one condition suffix that is **not** shared with Jump or Call: **IfIRQ** tests whether an interrupt request is pending. Because it does not make sense as a Jump or Call condition it appears only on the If command.

```
IfIRQ
    Call @HandleInterrupt
EndIf
```

10. Assembler directives

Directives control assembly-time behavior. They do not themselves emit ISA instructions, although some (like **Def** with an initializer) do emit data bytes into the output image. All directives are processed at assembly time.

10.1 Overview

Directive	Purpose
Def	Declare data storage or a symbolic constant (§10.2)
Section	Select the current output section (§10.3)
Include	Textually include another source file (§10.4)
Beg	Mark the program entry point (§10.5)
Origin	Set the assembly location counter (§10.6)
BegMacro / EndMacro	Define a macro (§10.7)
Export / Import	Multi-module linker visibility (§10.8)
AsmIf / AsmElse / AsmEndIf	Conditional assembly (§10.9)

10.2 Def — data and symbolic constants

All data declarations use the unified **Def** directive. The type keyword determines the data type. Sizes are in the **native unit of the type**: bytes for Binary and Integer, digits for Decimal, bits for Bit, and characters for Text.

Type keyword	Syntax	Description
Bin	Def <name> Bin <bytes> [#:<value>]	Unsigned binary. Size in bytes.

Type keyword	Syntax	Description
Int	Def <name> Int <bytes> [[:<value>]]	Signed ones-complement integer. Size in bytes.
Dec	Def <name> Dec <digits> [<positions>] [F] [[:<value>]]	Packed-decimal storage. <digits> 1-98. Optional <positions> 0-49 decimal places (default 0). Optional F marks float mode.
Bit	Def <name> Bit <bits>	Bit field, 1-2000 bits. Storage is $\text{ceil}(\text{bits} / 10)$ Kiss bytes. No initializer form.
Txt	Def <name> Txt <chars> [[:<value>]]	Text. Size in characters.
Sym	Def <name> Sym <type> <value>	Symbolic constant — compile-time substitution, no storage allocated. <type> is Bin, Int, Dec, or Txt.

The **#:** in a **Def** initializer omits the type letter because the type keyword already specifies it.

Def Dec — the float-mode flag

For fixed-point decimal storage, the **<positions>** argument fixes the number of fractional digits. Adding the literal keyword **F** after **<digits>** and the optional **<positions>** selects **float mode** — the value tracks its own decimal position within the packed-digit stream and may vary across stored values.

```
Def TaxRate Dec 10 4 #:8.25      // fixed, 10 digits, 4 positions
Def Rate    Dec 12 F           // float mode, 12 digits
Def Pi      Dec 20 18 #:3.14159265
```

Examples

```
// Binary and Integer
Def Counter    Bin 4           // 4 bytes, zero-filled
Def Flags      Bin 2 #:255     // 2 bytes, initialized to 255
Def OnHand     Int 4           // signed 4-byte integer

// Decimal
Def Total      Dec 15 2        // fixed, 15 digits, 2 positions
Def Balance    Dec 8 #: -67    // positions default to 0

// Bit and Text
Def InStock    Bit 1           // single-bit flag
Def Bitmap     Bit 137         // 137 bits -> 14 Kiss bytes
Def UserName   Txt 50          // 50 chars, zero-filled
Def AppTitle   Txt 30 #: "Kiss OS"

// Symbolic constants (no storage)
Def MaxUsers   Sym Int 5
Def Pi         Sym Dec 3.14159265
Def AppName    Sym Txt "Kiss OS"
Def DEBUG      Sym Bin 1       // for conditional assembly
```

10.3 Section — output sections

Directive	Syntax	Description
Section	Section <name>	Begin a named section.

Standard section names:

Section	Contents
Code	Executable instructions
Data	Read/write data storage
Const	Read-only data

The linker uses section names to organize the output binary. Sections can be entered and re-entered in any order; the assembler stitches same-named sections across files.

```

Section Code
@Main
    // ...

Section Data
Def Counter Bin 4

Section Code
    // more code

```

10.4 Include — textual inclusion

Directive	Syntax	Description
Include	Include "<filename>"	Include a source file at this point.

The included file is assembled as if its contents appeared inline. Nested **Includes** are permitted up to a maximum depth of **8** levels (per the Preprocessor implementation). Beyond that, the assembler reports a depth-exceeded error.

```

Include "macros.kasm"
Include "register-aliases.kasm"

```

10.5 Beg — program entry point

Directive	Syntax	Description
Beg	Beg @<entry-label>	Mark the program entry point.

Beg identifies where execution begins. The target label must be defined in the same assembly unit (or imported and resolved at link time). **Beg** replaces the older **End @<label>** form from earlier drafts.

```
Beg @Main
```

10.6 Origin — assembly location counter

Directive	Syntax	Description
Origin	Origin <address>	Set the assembly location counter to a specific Kiss byte address.

Default. If no **Origin** directive appears in a source file, the assembler's starting location counter is **10001**. The first 10,000 Kiss byte addresses (1-10000) are reserved for future boot / system use; ordinary user code begins at 10001.

Alignment. The target address must land on a 10-byte (one instruction) boundary counted from

the previous location counter. A misaligned target is a hard error — Kiss code is strictly 100-bit-aligned.

Forward-only. **Origin** may only move the location counter forward, or leave it where it is (equal to the current address is accepted as a no-op, useful for defensive assertions). An attempt to move the counter backward is a hard error, because backward jumps would overlap already-emitted code.

Gap filling. When **Origin** moves the location counter forward past already-emitted instructions, the gap is filled with zero-valued instruction words. Kiss reserves the all-zeros opcode as invalid, so any accidental execution of gap space produces an invalid-opcode trap rather than silent undefined behavior.

Multiple Origins. A single source file may contain any number of **Origin** directives, each subject to the forward-only rule relative to the current location counter.

Rejected forms.

- **Origin 0** — address 0 is reserved as invalid/unmapped in the 1-based address space.
- **Origin <addr>** where <addr> is less than the current location counter — backward move.
- **Origin <addr>** where the byte gap from the current counter is not a multiple of 10 — alignment error.
- **Origin** with no operand or with a non-numeric operand — syntax error.

Typical uses. Boot code expected at a specific entry address, memory-mapped regions, or any scenario where code must land at a hardware-mandated address.

Example:

```
// Default entry at 10001
@Main
    NOP
    Brk
Beg @Main

// Place a separate routine at 20001
Origin 20001
@Handler
    // ... handler code ...
    Retrn
Export @Handler
```

10.7 BegMacro / EndMacro — macro definitions

Directive	Syntax	Description
BegMacro	BegMacro <name> [<param1> ... <param4>]	Begin a macro definition. Up to 4 parameters.
EndMacro	EndMacro	End a macro definition.

A macro body is captured and expanded inline at each invocation. The parameter limit is a **hard**

maximum of 4; a **BegMacro** with more than four parameters is a syntax error.

See §12 for the full macro treatment, including label uniquification and nesting limits.

10.8 Export / Import — linker visibility

Directive	Syntax	Description
Export	Export @<label>	Make a label visible to the linker.
Import	Import @<label>	Declare a label as defined in another module.

Export and **Import** are the multi-module primitives. See §13 for the full multi-module story.

10.9 AsmIf / AsmElse / AsmEndIf — conditional assembly

Directive	Syntax	Description
AsmIf	AsmIf <cond>	Begin a conditional-assembly block.
AsmElse	AsmElse	Alternate branch.
AsmEndIf	AsmEndIf	End the block.

These directives are distinct from the runtime **If*** / **EndIf** instructions. They select whether a block of source is included at assembly time.

The condition on **AsmIf** is either:

- a single identifier — truthy test (non-zero numeric value, or non-empty text value), or
- **<lhs> <op> <rhs>** — a three-argument comparison, where **<op>** is one of:

Operator	Meaning
==	equal
!=	not equal
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal

Text and numeric operands may not be mixed in a single comparison.

```
Def DEBUG    Sym Bin 1
Def MaxUsers Sym Int 5

AsmIf DEBUG
    Call @DumpRegisters
AsmEndIf

AsmIf MaxUsers > 1
    Call @InitMultiUser
AsmElse
    Call @InitSingleUser
AsmEndIf
```

11. Predefined assembler constants

The assembler provides built-in named constants for documented fields of the **StatusFlags** alias (**SF**), **MACHINE_CONFIG**, **MODULE_CONFIG**, and **CORE_TYPE** registers. They are resolved at assembly time and cannot be redefined with **Def**.

11.1 Overview

- **Single-bit constants** resolve to a bit position. They work with **BitTstOn**, **BitTstOff**, **BitOn**, **BitOff**, and with the **If*** flag-testing instructions.
- **Multi-bit field constants** resolve to a (**start, width**) pair. They work with **BitRead** (extract a field into a register) and **BitWrite** (insert a value into a field).
- **Value constants** (such as **CORE_APP**) are numeric constants intended to be compared against the contents of a register read.
- **“Not” aliases** (such as **NZ**, **NPos**, **NEq**) instruct the assembler to emit the corresponding “test off” form rather than changing the underlying bit set.

11.2 StatusFlags flag constants

StatusFlags is the readable assembler alias for the architectural **SF** register. The Status Flags register holds three independent flag groups: Arithmetic (bits 1–7 and 22–23), Result-property (bits 8–12), and Compare (bits 13–21). Operations in one group do not clobber the others — an arithmetic op leaves Compare flags alone, a **Compare** leaves Arithmetic and Result-property alone, and **Test** writes Result-property + Compare without disturbing Arithmetic.

The Arithmetic group is intentionally non-contiguous: bits 1–7 hold primary arithmetic signals, while bits 22–23 hold the **THi**/**TLo** truncation-direction signals added after the first 21 flag bits were assigned.

Arithmetic flags — primary — set by arithmetic / logical operations carrying the **Set** modifier:

Constant	Bit	Meaning	Not-alias
Ovr	1	Mathematical result exceeded positive capacity	NOvr
Und	2	Mathematical result exceeded negative capacity	NUnd
Wrp	3	Modular wrap occurred (Add/Sub crossed capacity)	NWrp
Cry	4	Add result didn't fit (either direction)	NCry
Brw	5	Sub result didn't fit (either direction)	NBrw
Inf	6	Decimal result is $\pm$ infinity	NInf
AnyCarry	7	Decimal carry/borrow digit pending (chain math)	NAnyCarry

Ovr / **Und** are purely directional — they describe the mathematical direction of an out-of-range result, not which side of the digit/bit array was truncated. **Wrp** fires only for Add/Sub modular wrap. Mul/Div precision overflow uses **Ovr/Und** plus **THi**; Div nonzero remainder, Fixed-Dec

rescale, and right-shift precision loss use **TLo** without directional **Ovr/Und**.

Cry is set only after **Add** when the result didn't fit; **Brw** is set only after **Sub** when the result didn't fit. Both are direction-agnostic. Add never sets **Brw**; Sub never sets **Cry**. Internal arithmetic events that don't propagate to a final out-of-range result — for example **I.Sub** of **+50** and **+60** producing **-10**, which has an internal bit-level borrow but a representable result — do not fire **Brw** or **Cry**.

Arithmetic flags — truncation direction:

Constant	Bit	Meaning	Not-alias
THi	22	High-end digits/bits were dropped	NTHi
TLo	23	Low-end digits/bits were dropped	NTLo

THi / **TLo** describe which end of the result got truncated, independent of the mathematical direction signaled by **Ovr/Und**. Add/Sub use **Wrp** as the truncation-implies signal, so they do not set **THi/TLo**.

Result-property flags — set by arithmetic ops with **Set** and by **Test**:

Constant	Bit	Meaning	Not-alias
Z	8	Result is zero	NZ
Pos	9	Result is positive (Kiss reversed-sign: sign bit = 1)	NPos
Neg	10	Result is negative (sign bit = 0)	NNeg
EvN	11	Result is even	NEvN
Odd	12	Result is odd	NOdd

Compare flags — set by **Compare**, **ChkBnds**, and (when implemented) **CmpMag**:

Constant	Bit	Meaning	Not-alias
Eq	13	Operands equal	NEq
Gt	14	First > second (signed)	NGt
Ge	15	First ≥ second (signed)	NGe
Lt	16	First < second (signed)	NLt
Le	17	First ≤ second (signed)	NLe
Abv	18	First > second (unsigned magnitude) — <i>reserved</i>	NAbv
AbvE	19	First ≥ second (unsigned magnitude) — <i>reserved</i>	NAbvE
Blw	20	First < second (unsigned magnitude) — <i>reserved</i>	NBlw
BlwE	21	First ≤ second (unsigned magnitude) — <i>reserved</i>	NBlwE

Compare and **ChkBnds** write only **Eq** plus the signed directional quartet (**Gt/Ge/Lt/Le**). The magnitude directional quartet (**Abv/AbvE/Blw/BlwE**) is reserved for **CmpMag** (magnitude compare), which is **spec-defined but not yet implemented** — no current Kiss instruction writes those four bits, so the **Abv** / **Blw** condition suffixes always evaluate against **0** until **CmpMag** lands. The two directional quartets are disjoint, so a **Compare** followed by a **CmpMag** would leave both readable side by side.

11.3 MACHINE_CONFIG field constants

Single-bit feature flags:

Constant	Meaning
HAS_AI	AI core(s) present
HAS_DECIMAL_AI	D.MtxMulAdd supported on AI cores
HAS_VM	Virtual memory hardware present
HAS_HYPERVISOR	Level 2 hypervisor partitioning supported
HAS_OPTICAL	Optical interconnect present
HAS_TERMINAL	Terminal controller present

Multi-bit fields:

Constant	Meaning
AI_CORE_COUNT	Number of AI cores
APP_CORE_COUNT	Number of app cores
KERNEL_CORE_COUNT	Number of kernel cores
AI_ARRAY_ROWS	Systolic array row dimension
AI_ARRAY_COLS	Systolic array column dimension
MAX_USERS	Maximum simultaneous users
TERMINAL_COUNT	Number of terminal ports
MEM_CAPACITY	Total system memory (GB)
THREADS_PER_CORE	Threads per app core
IMPL_VENDOR	Vendor ID
IMPL_VERSION	Implementation version

11.4 MODULE_CONFIG field constants

Single-bit module flags:

Constant	Meaning
MOD_BASE	Base module present (always set)
MOD_DECIMAL	Decimal module present
MOD_SYSTEM	System module present
MOD_ATOMIC	Atomic module present
MOD_AI	AI/Tensor module present
MOD_USERINT	User Interrupts module present
MOD_DMA	DMA/Streaming module present
MOD_SIMD	Packed SIMD module present
MOD_TXMEM	Transactional Memory module present
MOD_CRYPT0	Crypto module present

11.5 CORE_TYPE value constants

These are numeric values (not bit positions) intended for comparison against the `CORE_TYPE` register:

Constant	Value	Meaning
CORE_APP	1	App core
CORE_KERNEL	2	Kernel core
CORE_AI	3	AI core
CORE_EMBEDDED	4	Embedded core

11.6 Usage examples

```
BitTstOn Cry StatusFlags          // test StatusFlags.Cry implicit-test context
BitOff  Ovr StatusFlags           // clear StatusFlags.Ovr
BitTstOn HAS_AI MACHINE_CONFIG    // test a hardware capability
BitRead APP_CORE_COUNT MACHINE_CONFIG R1A
                                   // extract app-core count into R1A
BitTstOn MOD_DECIMAL MODULE_CONFIG // check if Decimal module is present
BitTstOn MOD_ATOMIC MODULE_CONFIG  // check if Atomic module is present

I.Compare CORE_TYPE #B:CORE_KERNEL // branch on kernel-core check
IfEQ
    Call @InitKernelPath
EndIf
```

12. Macros

12.1 Definition

A macro is declared with `BegMacro` and closed with `EndMacro`. The body between them is captured verbatim and expanded at each invocation site.

A macro may declare **at most 4 parameters**:

```
BegMacro SwapRegs A B
    I.MoveL A R1ZA          // temp = A
    I.MoveL B A             // A = B
    I.MoveL R1ZA B          // B = temp
EndMacro
```

A macro body may itself contain macro invocations (see §12.5 on depth).

12.2 Invocation syntax

A macro is invoked the same way an instruction is written: the macro name followed by as many operand tokens as it takes parameters.

```
SwapRegs R1A R1B           // expands to the three I.MoveL lines above
```

12.3 Parameter substitution

Each parameter name inside the macro body is textually replaced with the corresponding operand supplied at the invocation. Substitution is positional and purely textual — the assembler does not distinguish register operands from literal operands at expansion time.

```
BegMacro BumpBy A Amount
    I.Add A Amount A
EndMacro

BumpBy R1A #I:5           // -> I.Add R1A #I:5 R1A
BumpBy Counter Step       // -> I.Add Counter Step Counter
```

12.4 Label unification

Labels defined inside a macro body are automatically rewritten on each invocation. The assembler appends a suffix of the form `_M<n>` to every intra-macro label, where `<n>` is a unique counter that increments with each expansion.

```
BegMacro Countdown Count
    @Loop
        I.Decr #I:1 Count
        JmpNZ @Loop
EndMacro

CountDown R1A      // @Loop -> @Loop_M1 ; JmpNZ @Loop_M1
CountDown R1B      // @Loop -> @Loop_M2 ; JmpNZ @Loop_M2
CountDown R1C      // @Loop -> @Loop_M3 ; JmpNZ @Loop_M3
```

Label references inside the same macro body resolve to the unquified form — `JmpNZ @Loop` inside `CountDown`'s body finds the rewritten label for the same invocation. **References from outside the macro body** do not see the unquified labels; intra-macro labels are **macro-local** and cannot be reached from caller code. External labels referenced from inside a macro (for example `@GlobalHandler`) are preserved unchanged.

12.5 Nesting depth limit

A macro may invoke another macro, and that macro may invoke a further macro, and so on. The Preprocessor enforces a maximum nesting depth of **16** expansions (`c_MaxMacroDepth`). Beyond that the assembler reports a macro-depth-exceeded error. In practice, straightforward layered helper macros stay well within this limit.

13. Multi-module linking

13.1 Overview of separate compilation

A Kiss program can be split across multiple source files that are assembled independently into object files and combined by the linker. This lets large programs be structured as libraries and lets incremental builds recompile only what has changed.

13.2 Export — making a label linker-visible

Export @<label> marks a label so that other modules can resolve references to it. Without **Export**, a label is module-local.

```
// File: math.kasm
Export @CalcInterest

@CalcInterest
    // ... implementation ...
    Retrn
```

13.3 Import — declaring an external label

Import @<label> declares that a label is **defined in some other module** and should be resolved by the linker. The assembler permits references to the imported label as if it were locally defined.

```
// File: main.kasm
Import @CalcInterest

@Main
    Call @CalcInterest
    Retrn

Beg @Main
```

13.4 .kobj object file format

Each source file is assembled into a **.kobj** object file that records its emitted code, data, section layout, export table, and import relocations. The linker combines **.kobj** files into an executable **.kexe**. The object file format is documented in [docs/linker-design.md](#); the assembler-facing surface is just the **Export/Import** pair above.

13.5 Entry point in the linked program

The **Beg @<label>** directive identifies the program's entry point. In a multi-module program, exactly one module supplies the **Beg** directive; the named label must resolve to a label exported by some module in the link set (or defined in the same module as **Beg**).

14. Preprocessor features

14.1 Overview

Kiss assembly has a small **preprocessor** that runs before the main assembly pass. The preprocessor is responsible for:

- textual file inclusion (**Include**)

- symbol definition (**Def Sym**)
- conditional assembly (**AsmIf/AsmElse/AsmEndIf**)
- macro capture and expansion (**BegMacro/EndMacro** and invocations)

After the preprocessor has finished, the assembler sees a single flat line stream with no **Includes**, no macro definitions, no conditional-assembly gates, and no macro invocations — only real instructions, directives, labels, and comments.

14.2 Include — textual inclusion

Include splices a file's lines into the current stream. Nested includes are permitted to a maximum depth of **8** levels. See §10.4.

14.3 AsmIf / AsmElse / AsmEndIf

Conditional-assembly directives include or exclude blocks of source based on symbol values. They support truthy single-identifier tests and three-argument comparisons using the **==**, **!=**, **<**, **<=**, **>**, **>=** operators. See §10.9.

14.4 Sym definitions as conditional-assembly predicates

Def Sym creates a compile-time symbolic constant with no storage. Such symbols are ideal as **AsmIf** predicates and tunable build parameters:

```
Def DEBUG          Sym Bin 1
Def MaxUsers       Sym Int 10
Def AppName        Sym Txt "Kiss OS"

AsmIf DEBUG
    Call @DumpRegisters
AsmEndIf

AsmIf MaxUsers >= 2
    Call @InitMultiUser
AsmEndIf
```

14.5 Macro expansion as a preprocessor pass

Macro invocations are recognized and expanded by the preprocessor. Parameter substitution and intra-macro label uniquification (§12.4) all happen before the main assembly pass sees the expanded text. This means macro-generated labels and instructions participate normally in everything the assembler does later — section tracking, **Export/Import**, conditional-assembly gating, and so on.

15. Complete examples

The examples in this section use `"..."` text literals to match the current Lexer implementation; the planned guillemet transition is discussed in Appendix B.

15.1 Hello-world-style program

Load a message, write it to the terminal port, and stop.

```
Section Data
Def HelloMsg  Txt 14 #:"Hello, Kiss!\n"

Section Code

@Main
  WriteL 14 Port1 @HelloMsg      // write 14 chars to terminal port
  Brk                               // halt
Beg @Main
```

15.2 Loop with conditional jump

Count down from 10 using the `Decr + JmpNZ` pattern. `I.DecrSet` would fold the flag update into a single instruction, but the plain form is clearer to start with.

```
Section Code

@Countdown
  SetVal R1A #I:10                // count = 10
@Loop
  WriteR 2 Port1 R1A              // display current count
  I.DecrSet #I:1 R1A              // count -= 1 and set flags
  JmpNZ @Loop                     // repeat while non-zero
  Retrn
Beg @Countdown
```

15.3 Conditional block

`If*` / `EndIf` run ISA instructions conditionally. The assembler-only `Else` keyword separates the true and false branches:

```
I.Compare R1A R1B
IfLT
  I.MoveL R1A R2A                // R1A was smaller
Else
  I.MoveL R1B R2A                // R1B was smaller or equal
EndIf
```

15.4 Subroutine with Call / Retrn

A caller invokes a subroutine with `Call`; the subroutine returns with `Retrn`. `Call` saves R1–R4, `SF`, the CEB state, and the special-register set to the protected shadow call stack. R5 is not saved — it is the parameter-passing bank.

```
Section Code
```

```

// Sum R5A + R5B, leaving the result in R5A (parameter-passing bank).
@AddTwo
    I.Add R5A R5B R5A
    Retrn

@Main
    SetVal R5A #I:7
    SetVal R5B #I:35
    Call @AddTwo                // R5A <- 42
    Brk
Beg @Main

```

15.5 Multi-module example

A library module exports a helper; a main module imports and calls it.

```

// File: lib.kasm
Section Code
Export @Double

@Double
    I.Add R5A R5A R5A           // R5A = R5A + R5A
    Retrn

// File: main.kasm
Section Code
Import @Double

@Main
    SetVal R5A #I:21
    Call @Double                // R5A <- 42
    Brk
Beg @Main

```

15.6 Macro definition and invocation

A macro that counts down a register and illustrates intra-macro label uniquification (§12.4):

```

Section Code

BegMacro Countdown N
    @Loop
        WriteR 2 Port1 N
        I.DecrSet #I:1 N
        JmpNZ @Loop
EndMacro

@Main
    SetVal R1A #I:3
    SetVal R1B #I:5
    Countdown R1A                // @Loop becomes @Loop_M1
    Countdown R1B                // @Loop becomes @Loop_M2
    Brk
Beg @Main

```

Each **CountDown** invocation produces its own unique **@Loop_M<n>** label, so the two countdowns do not collide.

16. Appendix A: EBNF grammar summary

The following EBNF covers the main source-line shapes the assembler accepts. It is a compact summary, not a full formal grammar; for the working reference see

<docs/assembler-language/kiss-assembler-grammar.md>.

```
(* Top-level structure *)
line                = label_line | instruction_line | directive_line
                    | line_comment | block_comment | blank_line ;
label_line          = label [ ws line_comment ] ;
instruction_line    = instruction [ ws line_comment ] ;
directive_line      = directive [ ws line_comment ] ;

(* Labels *)
label               = "@" identifier ;
target_label        = "@" identifier ;

(* Instructions *)
instruction          = mnemonic [ ws operand_list ] ;
operand_list        = operand { ws operand } ;
operand             = bare_numeric | typed_literal | register
                    | target_label | symbol ;
mnemonic            = identifier { "." identifier } ;

(* Literals *)
bare_numeric        = numeric_text ;
typed_literal       = "#" type_tag ":" literal_value ;
type_tag            = "B" | "I" | "D" | "T" | "" ; (* "" only valid in Def *)
literal_value       = quoted_string | unquoted_token ;
quoted_string       = "'" { any_char - newline } "'" ;

(* Directives *)
directive           = def_directive
                    | section_directive | include_directive | beg_directive
                    | origin_directive
                    | begmacro_directive | endmacro_directive
                    | export_directive | import_directive
                    | asmif_directive | asmelse_directive | asmendif_directive ;

type_keyword        = "Bin" | "Int" | "Dec" | "Bit" | "Txt" | "Sym" ;
sym_type_keyword    = "Bin" | "Int" | "Dec" | "Txt" ;
def_value           = "#:" ( quoted_string | expression ) ;
dec_positions       = integer ;
dec_float           = "F" ;

def_directive       = "Def" ws identifier ws type_keyword ws def_body ;
def_body            = sym_body | bin_body | int_body | dec_body
                    | bit_body | txt_body ;
sym_body            = "Sym" ws sym_type_keyword ws expression_or_string ;
bin_body            = "Bin" ws integer [ ws def_value ] ;
int_body            = "Int" ws integer [ ws def_value ] ;
dec_body            = "Dec" ws integer [ ws dec_positions ] [ ws dec_float ]
                    [ ws def_value ] ;
bit_body            = "Bit" ws integer ;
txt_body            = "Txt" ws integer [ ws def_value ] ;

section_directive   = "Section" ws identifier ;
include_directive   = "Include" ws quoted_string ;
beg_directive       = "Beg" ws target_label ;
origin_directive    = "Origin" ws expression ;
```

```

begmacro_directive = "BegMacro" ws identifier { ws identifier } ; (* max 4 params *)
endmacro_directive = "EndMacro" ;
export_directive   = "Export" ws target_label ;
import_directive   = "Import" ws target_label ;
asmif_directive    = "AsmIf" ws asmif_cond ;
asmelse_directive  = "AsmElse" ;
asmendif_directive = "AsmEndIf" ;
asmif_cond         = identifier
                    | asmif_term ws asmif_op ws asmif_term ;
asmif_op           = "==" | "!=" | "<" | "<=" | ">" | ">=" ;
asmif_term         = identifier | integer | quoted_string ;

(* Comments *)
line_comment      = "//" { any_char - newline } ;
block_comment     = "/*" { any_char } "*/" ;
ws                = space_or_tab { space_or_tab } ;

```

Notes on the sketch:

- The "A" and "PD" type tags are omitted from `type_tag` on purpose. #A: is simulator-only helper syntax and #PD: is deprecated — neither belongs to the current documented assembler surface.
- `Else` (the assembler keyword that separates the branches of an `If*/EndIf` block) is recognized contextually and is not listed as a directive.
- `BegMacro` is limited to 4 parameter names (§10.7, §12.1).

17. Appendix B: Character set and delimiter notes

Current text-literal delimiters

The current Lexer uses double-quote delimiters for text literals:

```

#T:"Hello, world"
Def AppTitle Txt 30 #:"Kiss OS"

```

Supported backslash escapes are `\`, `\\`, `\n`, and `\t`. Any other `\x` sequence produces the literal character `x`. A quoted string may not span a line.

Upcoming transition to guillemets

The 2026-03-29 architecture decision adopts **guillemet delimiters** for text literals:

```

#T:«Hello, world»
Def AppTitle Txt 30 #:«Kiss OS»

```

The Kiss character map reserves positions **47** (`«`) and **48** (`»`) for these delimiters. The Lexer has not yet been updated to accept the new form, so the examples throughout this manual use the current `"..."` delimiter. When the transition lands the change is intentionally **source-incompatible** — existing `"..."` text-literal sources will need to be converted.

The migration is planned as a coordinated update across:

- the assembler Lexer (accept «...» as the quoted-string form),
- the Preprocessor (text stripping when reading string-valued **Def Sym**),
- the assembler and engine test suites,
- the manuals and grammar reference,
- the VS Code extension grammar.

Until that coordinated update ships, programmers should keep using the quoted form. This appendix is the single place the transition is flagged; the rest of the manual intentionally shows only the currently-accepted syntax.

Character-set note

Kiss uses 10-bit character codes (one Kiss byte), which is enough for 1,024 code points. The character map defines the ASCII-like core, the guillemets at positions 47–48, and a generous reserve for future extensions. A full listing of the character map is maintained separately alongside the ISA reference.